Sprint 2 Presentation

Data Weaver - Distributed Agentic Data Pipeline - Capstone Project

Team - 2

Sprint 3

Agenda

- Team Member Roles and Responsibilities
- Problem Statement
- Project Description
- Team working agreement
- Personas
- MVP
- Technologies & Algorithms
- Diagrams
  - Architecture Diagrams
  - Context Diagram
  - Sequence Diagram
  - State Diagrams
  - Class Diagrams
- Sprint 2 Recap
- Product Backlog
- Sprint 3 Backlog
- Test Cases
- Metrics
  - Team velocity
  - Burndown Charts
  - Completed/Committed Ratio
- Retrospective
- Demo

Team Members & Roles

Tenzing Salaka

AI Engineer - Engineer Agent

Shreya Anand Kuvalekar

Scrum Master/UI Developer/System Architect

Orges Velia

AI Engineer-Architect Agent

Priya Prajapati

Backend & Data Engineer

Kushwanth Reddy Nomula

Frontend / Dashboard Engineer

Meghana Ventrapragada

Backend & Data Engineer

Problem Statement

Organizations accumulate fragmented, inconsistent, and poorly documented datasets across departments, resulting in “data swamp” conditions that hinder analytics and decision-making. Integrating such datasets requires significant manual effort to infer schema relationships, identify primary and foreign keys, resolve type mismatches, and validate joins.Existing LLM-based data tools attempt automation but rely on probabilistic reasoning without deterministic safeguards, often hallucinating schema mappings or producing unsafe joins that corrupt downstream analytics. Additionally, most solutions lack distributed scalability and structured separation between semantic reasoning and execution logic.There is a need for a reliable, distributed, self-correcting AI system that separates semantic modeling from deterministic data execution while enforcing invariant-based validation to ensure trustworthy data integration.

Project Description

Project Name:	Data Weave
Team:	Team 2
Project Description:	For data analysts, data engineers, and organizations dealing with fragmented datasets who struggle to clean, integrate, and understand inconsistent and unstructured data sources the Distributed Agentic Data Pipeline is a multi-agent, AI-driven data integration system using a distributed architecture that automatically maps, cleans, validates, and merges datasets while generating reliable data models and documentation Unlike traditional ETL tools or LLM-based solutions that rely on manual configuration or produce unreliable, unverifiable outputs Our application uses a self-correcting two-agent architecture with deterministic validation checks to ensure accuracy, scalability, and trustworthy data transformation
Benefit Outcomes:	• Better data integration decisions with AI-assisted schema inference and validation checks • Reduced ETL troubleshooting time through automated mapping, profiling, and transformation previews • Improved trust through transparent logs, ERD visibility, and human approval checkpoints • Faster delivery of analytics-ready datasets and data dictionaries for downstream reporting
GitHub Link:	https://github.com/htmw/S2026A-Team2

Team Working Agreement

Team Principles

- Kindness, respect, and patience in all interactions
- Clear ownership and responsibilities
- Even workload distribution; break large tasks
- Document attempted solutions before escalating
- Use GitHub for all updates and reviews

Checklist to Mark Task Complete

- Code runs with no syntax/runtime errors
- Meets acceptance criteria and tests
- Peer review and multi-member verification
- Push to branch and open PR with docs updated
- All relevant documentation tagged and reviewed

Alex Martinez — Senior Data Engineer
Industry: E-commerce / Healthcare / FinTech
Experience: 5–8 years

Description

It is designed to streamline the integration of fragmented legacy data sources by automating schema interpretation and relationship mapping. It introduces deterministic validation checks—such as row-count reconciliation and null-threshold monitoring—to ensure transformation accuracy. The platform also offers end-to-end transparency into data processing steps and produces downloadable, analysis-ready datasets along with comprehensive data dictionaries.

Goals

- Consolidate multiple unstructured CSV exports originating from legacy systems
- Enable safe, accurate joins while maintaining schema consistency
- Minimize time spent on manual ETL troubleshooting
- Provide dependable, analytics-ready datasets to downstream teams

Challenges

Non-standardized column naming conventions (e.g., *cust_id*, *customerID*, *user_ref*)

- Lack of documentation for legacy data structures
- Join amplification leading to inflated or duplicated row counts
- Undetected schema mismatches corrupting downstream dashboards
- Excessive time spent validating transformations rather than developing pipelines

Priya Sharma — BI Analyst
Industry: Retail / SaaS
Experience: 3–5 years

Description

This system empowers analytics and reporting teams to independently prepare and understand integrated datasets. It provides visual entity-relationship diagrams (ERDs) to clarify how tables connect, along with transparent pipeline logs that document validation steps. Users can download analysis-ready datasets and leverage auto-generated data dictionaries to better interpret column meanings—ensuring confidence in every output produced.

Goals

- Rapidly merge datasets to support reporting needs
- Gain clear visibility into table relationships through ERD views
- Access clean, dependable data without relying on engineering teams
- Build dashboards powered by accurate, trustworthy metrics

Challenges

- Delays caused by dependence on data engineering for cleaned datasets
- Ambiguity around primary and foreign key columns
- Duplicate records distorting KPI and metric calculations
- Insufficient metadata and lack of formal data dictionaries
- Skepticism toward AI-generated joins without validation guarantees

Daniel Kim — Chief Technology Officer / Head of Data

Industry: SaaS / FinTech / Healthcare

Experience: 10+ years

Description

This system is built to deliver enterprise-grade data reliability by combining deterministic validation with scalable AI infrastructure. It introduces invariant-based checks to ensure transformation accuracy, while maintaining full audit trails for governance and compliance. The architecture is designed for distributed, production-ready deployment and automatically generates documentation—including ERDs and data dictionaries—so organizations can trust, verify, and operationalize AI-driven data outputs at scale.

Goals

- Maintain consistent, high-quality data across all departments
- Lower engineering effort required for complex data integrations
- Strengthen governance, traceability, and audit readiness
- Implement scalable, AI-enabled infrastructure
- Showcase innovation through distributed AI system adoption

Challenges

- Data inconsistencies leading to conflicting reports across departments
- Significant engineering costs tied to building and maintaining ETL pipelines
- Limited schema transparency and insufficient documentation
- Concerns around AI systems generating unverified or non-auditable outputs
- Legacy or single-node infrastructure that cannot scale with enterprise demands

Minimum Viable Product

- Ingestion & Scout: Upload CSV/API via Stream lit UI; profiling, schema sniffing, PK/FK hints.
- Architect: Schema inference, ERD creation, data dictionary generation.
- Engineer: Generates Polars/Pandas code, cleaning steps, validation rules.
- Router + LLM Retry Loop: Auto-repair failed runs, regenerate improved transformations.
- Invariant Enforcement: Deterministic checks; halt + logs on violation.
- Loader & Storage: Successful pipeline saved to SQLite with logs + lineage.
- Dashboard: Stream lit UI shows ERD, profiling, invariants, logs, downloads.

Technologies & Algorithms

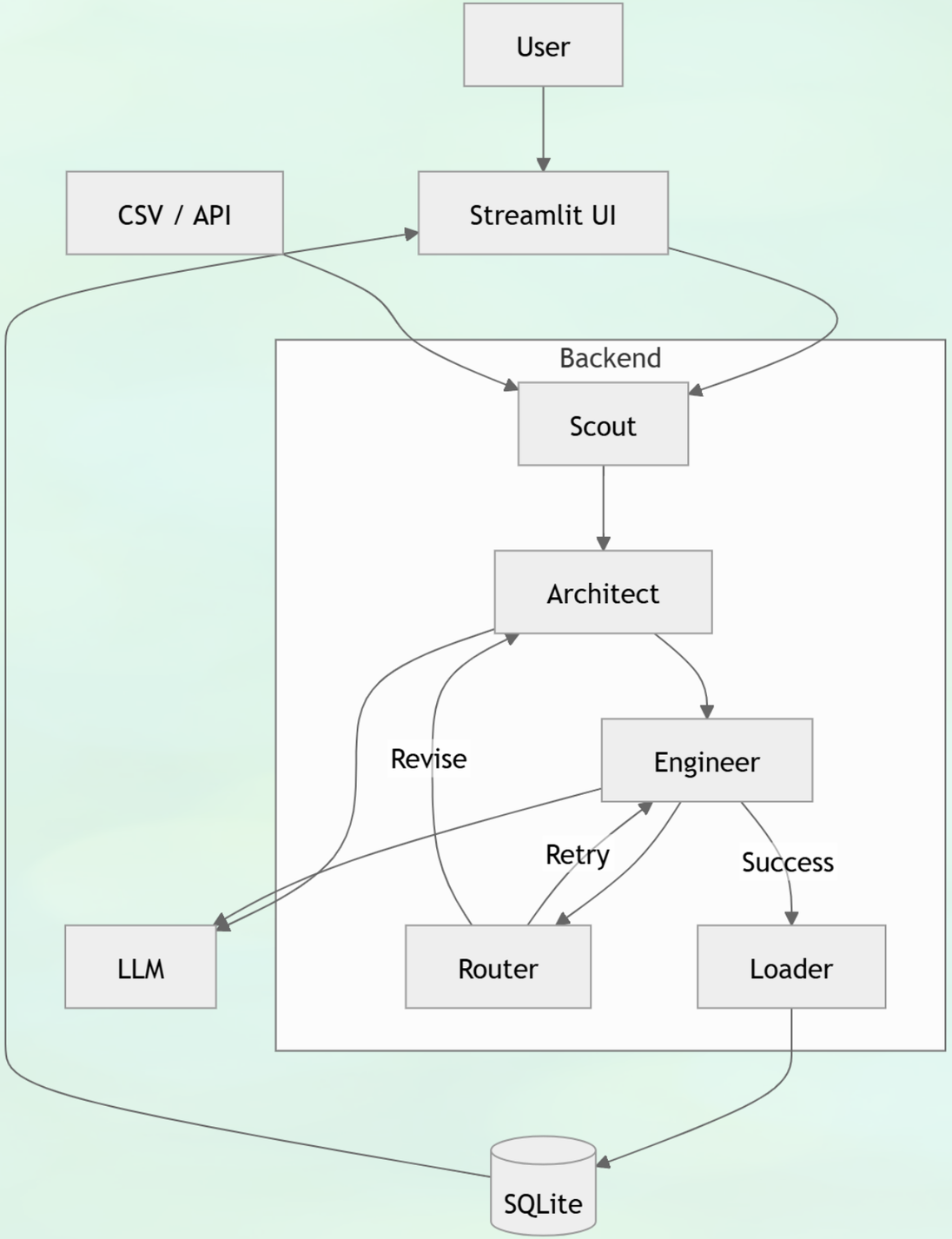
- **Architecture & Orchestration:** LangGraph — multi-agent cycles with explicit checkpoints
- API Layer: FastAPI for inter-node communication and control

- Networking: Tailscale for secure distributed connectivity

- UI: Streamlit dashboard for visualization & human checkpoints
- **Models:** local gpt-oss:20b (quantized) — Mapper high-reasoning, Aggregator high-throughput

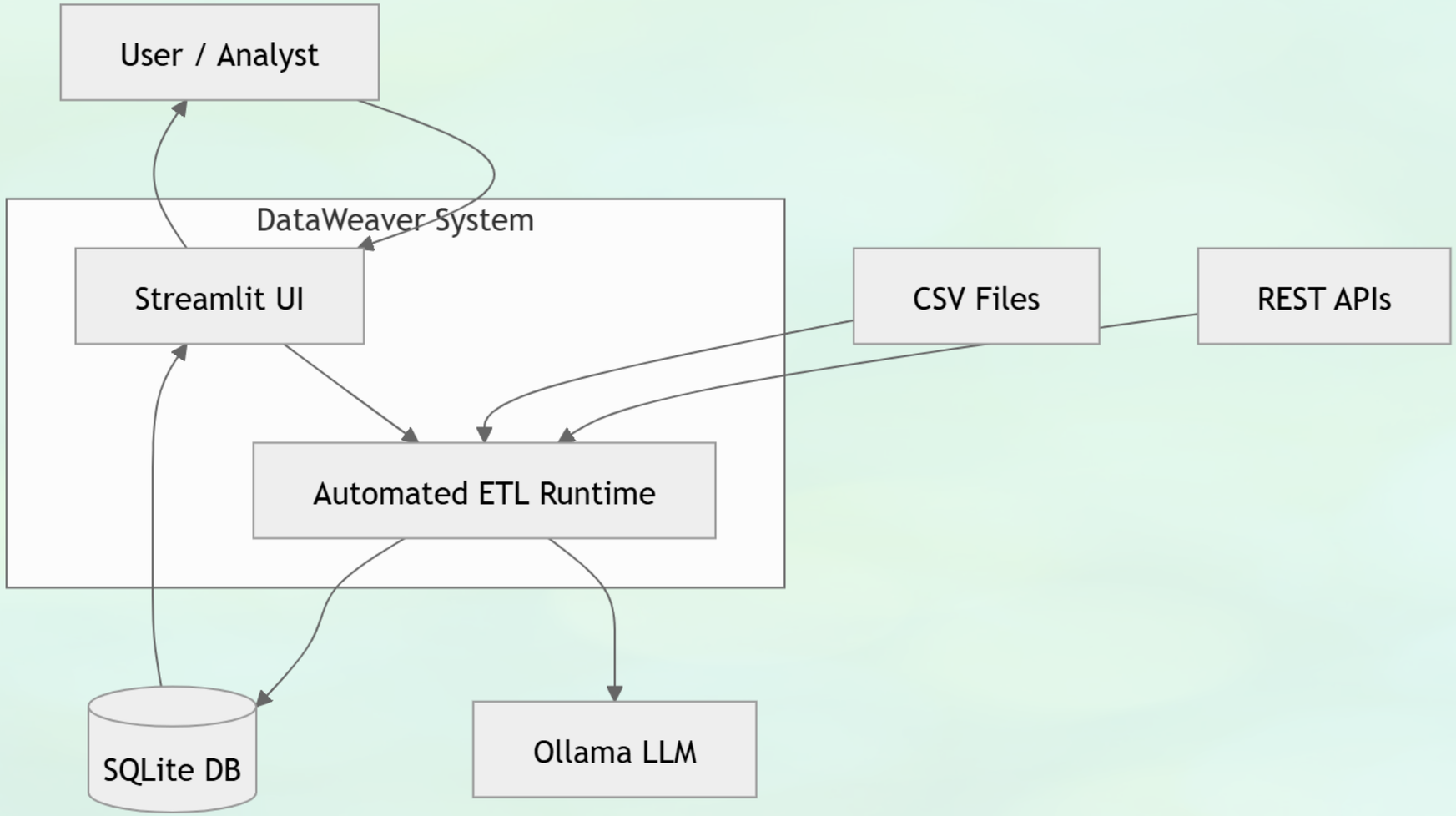
- **Data Processing:** Polars / Pandas (GPU-accelerated when available)

Architecture Diagram

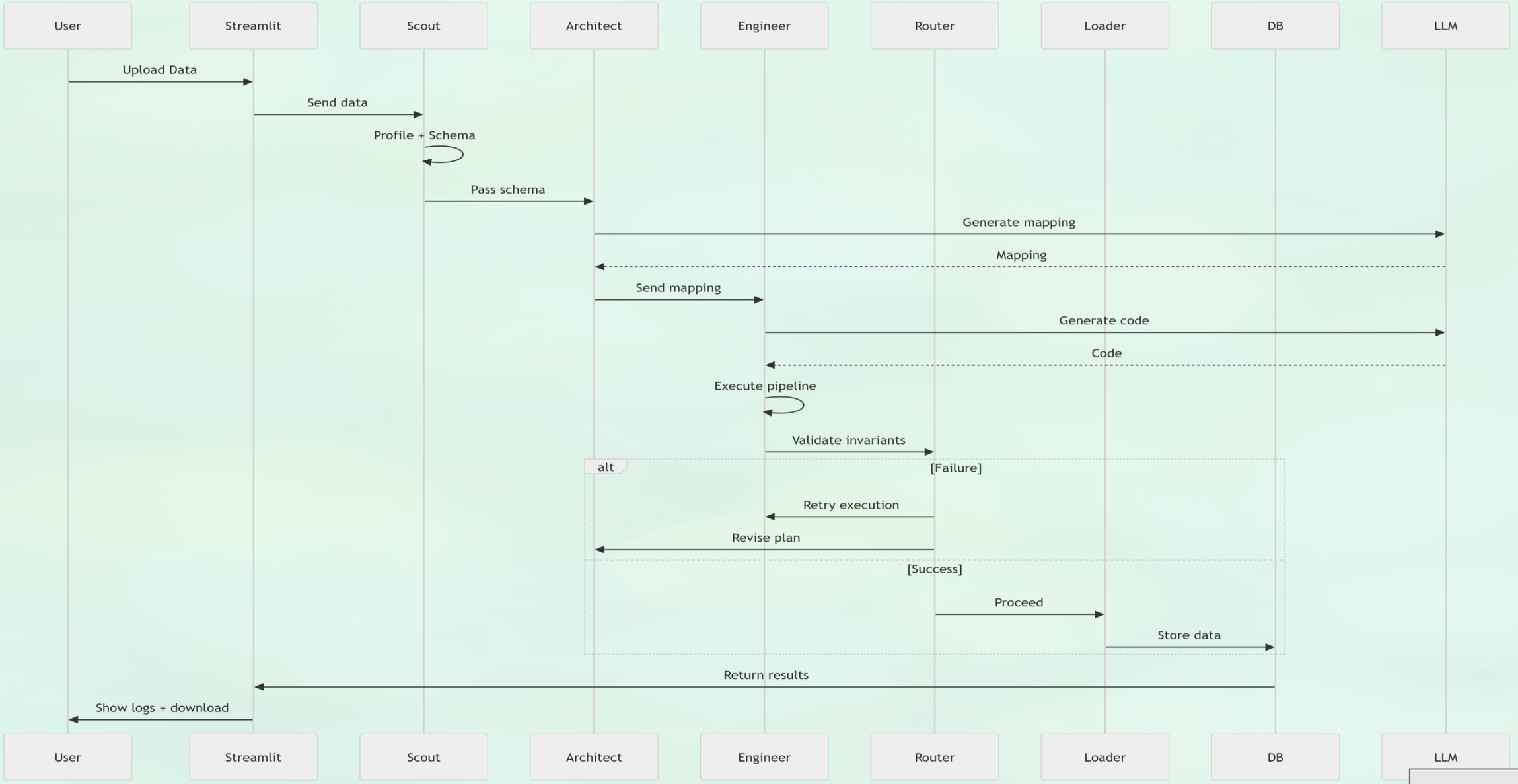


Sprint 3

Context diagram

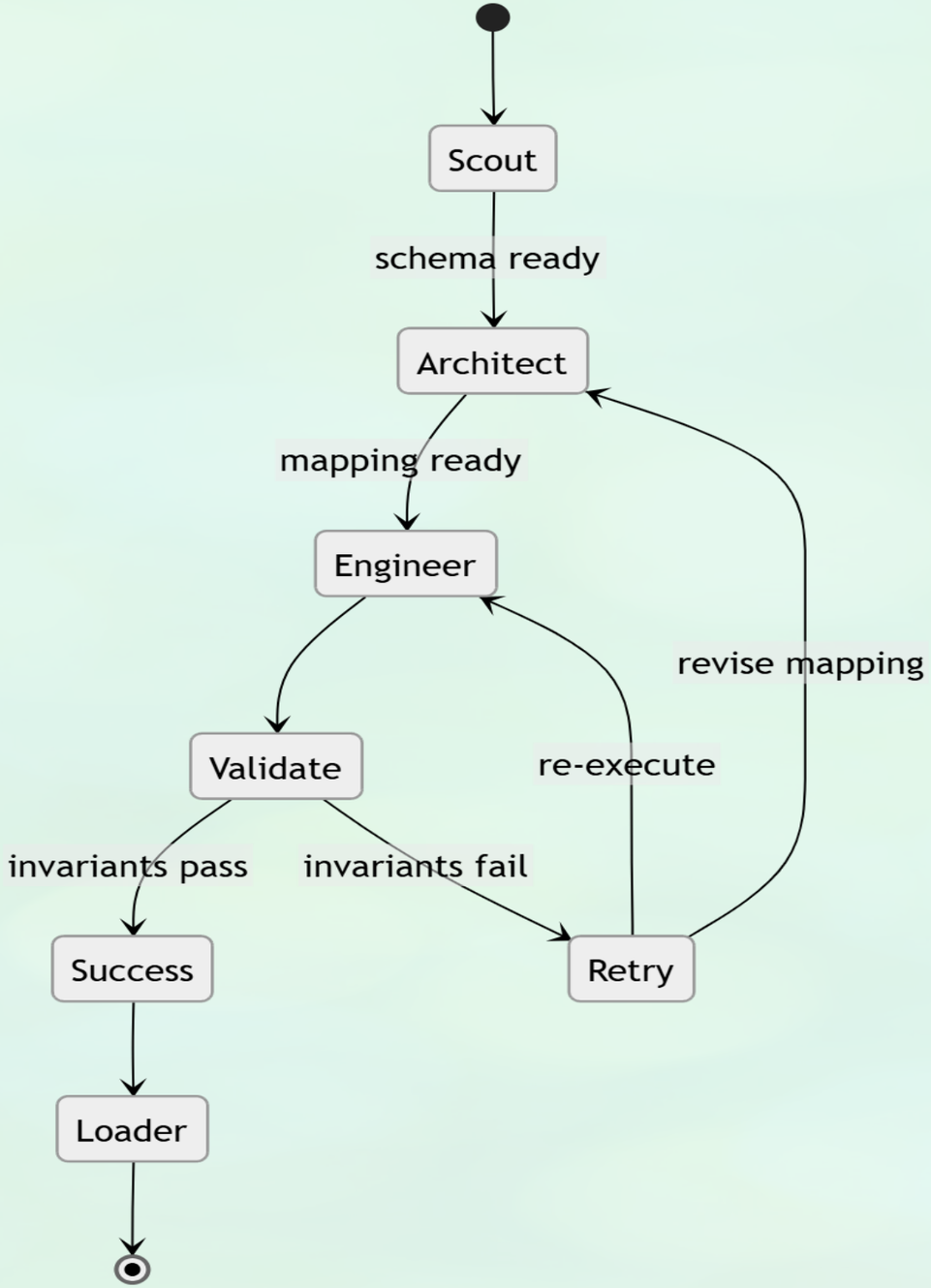


Sequence Diagram

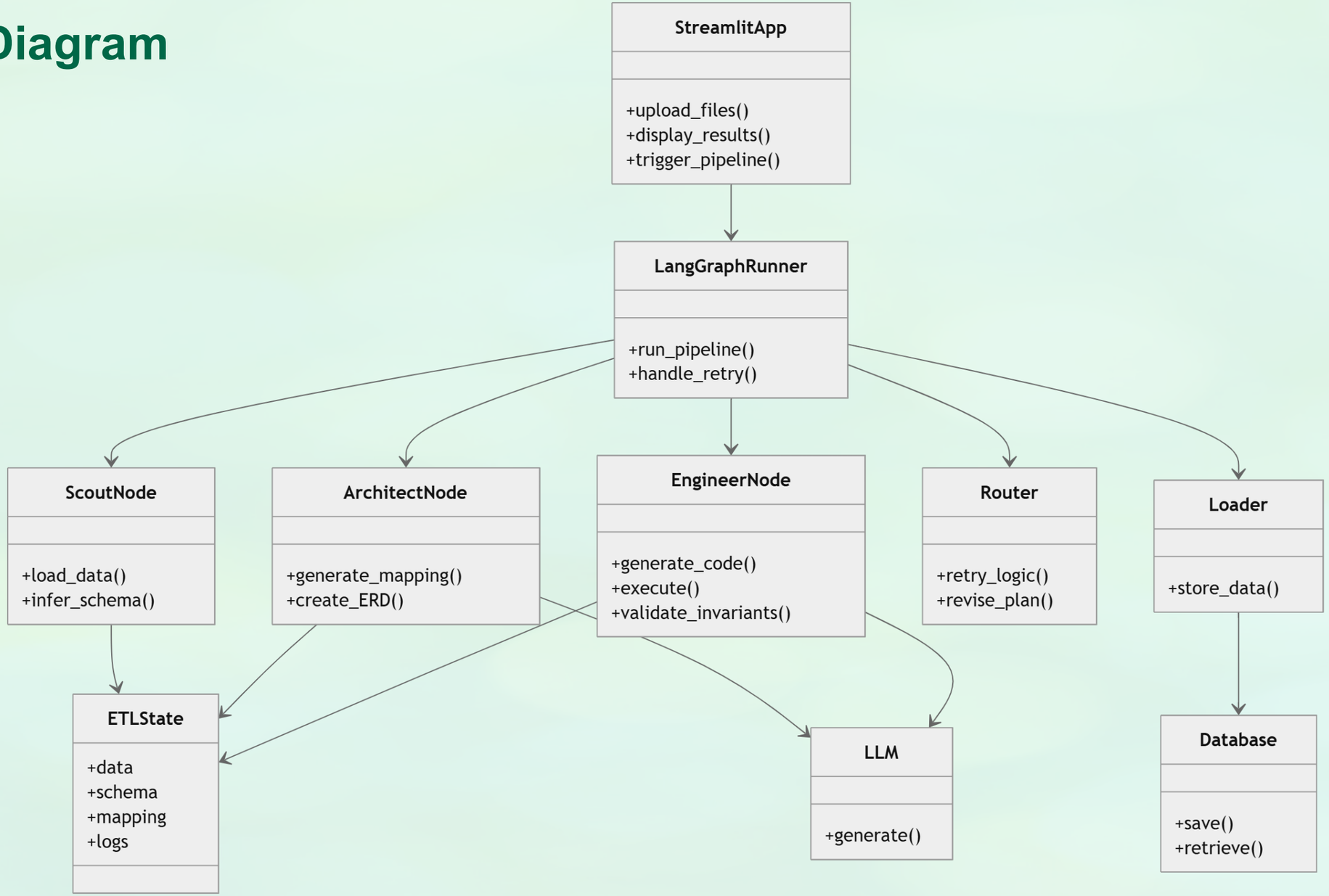


Sprint 3

State Diagram



Class Diagram



Sprint 2 - Recap

The primary objective of Sprint 2 was to improve the robustness of the distributed agentic pipeline through advanced validation checks, monitoring systems, and enhanced export/documentation capabilities.

Major Features Completed

- **US_006 – Row Count Validation**

Implemented invariant-based row count validation to prevent unsafe joins and detect data amplification issues during transformations. This ensured that merged datasets maintained logical consistency and reduced the risk of duplicated or inflated records.

- **US_007 – Schema Validation**

Developed schema verification mechanisms to confirm that transformed datasets matched expected target schemas. This improved downstream compatibility and reduced errors caused by datatype mismatches.

- **US_008 – Null Detection**

Integrated null-threshold monitoring to identify spikes in missing values across transformed datasets. The feature enabled early detection of data quality degradation and improved pipeline reliability.

Sprint 2 - Recap

- **US_009 – Pipeline Monitoring**

Built real-time monitoring and execution tracking capabilities for the distributed pipeline. Users could now visualize processing stages, validation checkpoints, and execution logs through the Streamlit dashboard.

- **US_010 – Data Download**

Added downloadable output functionality, allowing users to export cleaned and transformed datasets directly from the platform for analytics and reporting purposes.

Key Outcomes

- Improved trustworthiness of AI-driven data transformations
- Increased transparency through monitoring and validation logs
- Reduced risk of schema inconsistencies and incorrect joins
- Enhanced user accessibility with downloadable outputs
- Strengthened foundation for distributed processing capabilities planned in Sprint 3

Product Backlog

ID	Assigned	Name	Type	Creation Date	US	Size	Priority	Dependencies
US1	Orges/Tenzing	API & CSV Tooling	F	2025-03-11	As a data engineer, I want the agent to have tools to connect to APIs and read CSV files, so that I can start extraction.	Small	High	None
US2	Orges/Tenzing	Data Extraction & Transformation	F	2025-03-11	As a data engineer, I want the agent to extract and transform data from APIs and CSVs, so I can integrate sources.	Small	High	US1
US3	Orges/Tenzing	Sensitive Data Redaction	F	2025-03-11	As a compliance officer, I want the agent to redact sensitive data, so we ensure privacy.	Medium	Medium	US2
US4	Orges/Tenzing	ETL Step Verification	F	2025-03-11	As a data steward, I want to verify each ETL step, so we ensure quality.	Small	High	US2
US5	Orges/Tenzing	Agent Coordination Framework	F	2025-03-11	As a team lead, I want a coordination framework, so agents collaborate on tasks.	Medium	Medium	None
US6	Orges/Tenzing	Parallel Multi-Agent Execution	F	2025-03-11	As a team lead, I want multiple agents to collaborate in parallel, so complex tasks finish faster.	Large	Medium	US5
US7	Shreya/Kush	Schema Change Monitoring	F	2025-03-11	As a data steward, I want to monitor dataset headers, so we catch schema changes.	Medium	Medium	US2
US8	Shreya/Kush	Pre-Join Data Sampling	F	2025-03-11	As a product owner, I want the agent to sample data before joining, so only clean data is used.	Medium	Medium	US2
US9	Orges/Tenzing	Conversational Memory	F	2025-03-11	As a user, I want the agent to remember past requests, so interactions are contextual.	Large	Low	Later enhancement
US9.1	Priya/Meghana	Pipeline Performance	NF	2025-03-11	As a data engineer, I want the ETL pipeline to complete a standard extraction and transformation job within a reasonable time threshold, so that the system is practical for real workloads and not just demos.	Medium	Medium	US1

Sprint 3

Product Backlog

12	US10	Priya/Meghana	Fault Tolerance	NF	2025-03-11	As a data engineer, I want the pipeline to gracefully handle agent failures without crashing the entire run, so that a single broken step doesn't require restarting the whole job from scratch.	Medium	High	US1
13	US11	ongoing	Credential & API Key Security	NF	2025-03-11	As a compliance officer, I want API keys and database credentials to be stored securely and never logged in plain text, so that sensitive configuration data is not exposed in traces or outputs.	Small	High	US1
14	US12	ongoing	Model Resource Allocation	NF	2025-03-11	As a data engineer, I want LLM-based agents to be assigned to the appropriate GPU based on model size and VRAM requirements, so that the pipeline runs efficiently without memory errors.	Small	High	None
15	US13	Priya/Meghana	Pipeline Maintainability	NF	2025-03-11	As a developer, I want each agent to be modular and independently testable, so that changes to one agent don't require refactoring the entire pipeline.	Medium	Medium	None
16	US14	ongoing	Traceability & Auditability	NF	2025-03-11	As a data steward, I want every agent action and decision to be logged with a timestamp and input/output summary, so that the pipeline can be audited after a run completes.	Small	High	US6
17	US15	sprint 3	Scalability	NF	2025-03-11	As a data engineer, I want the pipeline to handle datasets of increasing size without significant performance degradation, so that it remains viable beyond small demo datasets.	Large	Low	Later enhancement
18	US16	Shreya/Kush	Error Detection During Extraction/Transformation	F	2026-03-11	As a user, I want the system to detect and notify errors during data extraction or transformation so that I can correct issues before processing continues.	Medium	High	Data processing engine, validation rules

Sprint 3

Product Backlog

19	US17	Shreya/Kush	Multi-Format Output Export	F	2026-03-11	As a user, I want to export processed data in formats like CSV, Excel, Google Sheets, and JSON so that I can use the data in other tools.	Medium	High	File export service, Google Sheets API
20	US18	Shreya/Kush	Drag & Drop / API / Link Upload	F	2026-03-11	As a user, I want to upload files via drag & drop or provide APIs/links in a textbox so that I can easily ingest data from different sources.	Large	High	uploader, API connector service
21	US19	Shreya/Kush	Database Preview Before & After Processing	F	2026-03-11	As a user, I want to preview the database before and after processing in table or ERD format so that I can understand the structure and transformations applied.	Large	Medium	Database visualization library, transformation engine
22	US20	Shreya/Kush	Basic Analytics Functions	F	2026-03-11	As a user, I want basic analytics like average, median, and summary stats so that I can quickly analyze my dataset without external tools.	Medium	Medium	Data processing engine, aggregation functions
23	US21	Shreya/Kush	Dropdown Filters for Data	F	2026-03-11	As a user, I want dropdown filtering options so that I can easily filter datasets and focus on relevant information.	Small	Medium	UI filtering components, dataset query layer
24	US22	ongoing	Data Quality Validation	F	2026-03-11	As a user, I want the system to validate dataset quality so that I can detect missing values, duplicates, and invalid data before processing. **Acceptance Criteria** Detect missing values Detect duplicate rows Validate column data types Generate validation summary	Medium	Medium	
25	US23	sprint 3	Dataset Health Score	NF	2026-03-11	As a user, I want the system to generate a dataset health score so that I can quickly understand the overall quality of my dataset. **Acceptance Criteria** Health score calculated from validation metrics Score range from 0–100 Health classification displayed (Healthy, Moderate, Poor) Score displayed in dataset report	Small	Medium	

Sprint 3

Product Backlog

						that I can understand how the system processed my dataset. **Acceptance Criteria** Each agent action is logged Timeline shows sequence of agent operations Each step displays agent name and action Timeline available in final dataset report			
26	US24	Priya/Meghana	Agent Decision Timeline	F	2026-03-11		Small	Medium	
27	US25	Shreya/Kush	Dataset Profiling	F	2026-03-11	provide summary statistics so that I can quickly understand the structure and distribution of my data. **Acceptance Criteria** System detects number of rows and columns System identifies column data types System calculates summary statistics for numeric columns System identifies categorical column distributions Profiling report generated	Medium	Medium	
28	US26	sprint 3	Sandbox Dry-Run Mode	F	13-03-2026	As a data engineer, I want the agent to support a sandbox dry-run mode on a small sample of the data so that I can see all planned transformations and health checks before committing changes to the full dataset.	Medium	Medium	
29	US27	sprint 3	Auto-Generated Run Docs	F	13-03-2026	As a product owner, I want the agent to generate human-readable documentation for each ETL run so that stakeholders can quickly understand what inputs, transformations, and outputs were involved.	Small	Medium	US26,US25
30	US28	sprint 3	User Feedback & Rating Loop	F	13-03-2026	As an end user, I want to rate pipeline runs and leave feedback so that the agent can suggest improvements to mappings, validations, or configurations over time.	Small	Medium	US27

Sprint 3

Sprint 3 Backlog

1	No.	Description	Acceptance Criteria	Feature	Story points	Priority	Assignee
2	US_S2_01	As a data steward, I want to validate and monitor data quality across the ETL pipeline so that errors are detected early and data integrity is maintained.	* ETL step execution logs are captured for each stage * Row-level errors are detected and clearly reported * Schema validation ensures input and output consistency * Null values and anomalies are identified correctly * Validation reports summarize all detected issues * API returns validation results in structured format * System prevents invalid data from progressing in pipeline	Data Validation & Quality	55	High	Tenzing / Orges
3	US_S2_02	As a system, I want agents to coordinate and execute tasks in parallel so that ETL performance and scalability improve.	* ETLState contains all required shared fields * Agents execute tasks in correct workflow order * Parallel execution runs without data conflicts * Retry mechanism works correctly for failed steps * Agent logs capture execution details and decisions * Pipeline execution completes successfully under coordination * System handles partial failures without crashing entire flow	Multi-Agent System	55	Medium	Tenzing / Orges
4	US_S2_03	As a user, I want flexible data ingestion methods so that I can easily upload datasets from multiple sources.	* File upload supports drag & drop functionality * API-based upload accepts valid file formats (CSV, JSON) * Upload progress is displayed to the user * Invalid files are rejected with clear error messages * Uploaded datasets are stored successfully * System handles large file uploads without failure	Data Input	34	High	Shreya / Kushawnth
5	US_S2_04	As a user, I want to export processed data in multiple formats so that I can use it externally.	* Processed data can be exported in CSV format * Processed data can be exported in JSON format * Processed data can be exported in Excel format * Download API returns correct file response * Exported data maintains correct schema and formatting * System handles export errors gracefully	Data Output	34	High	Tenzing / Orges

Sprint 3 Backlog

6	US_S2_05	As a user, I want to preview data before and after processing so that I understand transformations clearly.	* Dataset preview displays tabular data correctly * Schema view shows column names and data types * ERD visualization represents dataset relationships * Before and after data comparison is available * UI loads data efficiently without performance issues * API returns preview data accurately	Data Visualization	34	Medium	Shreya / Kushawnth
7	US_S2_06	As a user, I want basic analytics and filtering tools so that I can quickly analyze datasets.	* System calculates average and median correctly * Summary statistics are generated for datasets * Backend supports filtering queries * Aggregation results are returned in structured format * Queries execute efficiently on large datasets * Errors are handled and reported clearly	Analytics	34	Medium	Tenzing / Orges
8	US_S2_07	As a compliance officer, I want sensitive data to be redacted so that user privacy is protected.	*Schema changes are detected between pipeline runs *System identifies added, removed, or modified fields *Alerts are generated for unexpected schema changes *Schema change logs are stored for auditing *Monitoring integrates with ETL pipeline execution	Security	21	Medium	Tenzing / Orges

Sprint 3 Stories Completed

1	No.	Description	Acceptance Criteria	Priority	Feature	Story point	Assignee
2	US_S3_01	As a data engineer, I want the ETL pipeline to complete extraction and transformation within a reasonable time threshold so that the system is practical for real-world workloads.	* Pipeline completes within defined time threshold * Performance improves compared to previous runs * Bottlenecks are identified and optimized * Execution metrics are logged * Data accuracy remains unaffected	High	Pipeline Reliability & Performance	11	Orges Velia
3	US_S3_02	As a compliance officer, I want API keys and database credentials to be stored securely so that sensitive data is not exposed.	* Credentials are not hardcoded * Sensitive data is not logged * Secure storage mechanism is used * Access is restricted properly * Data exposure risks are minimized	High	Security & Resource Optimization	5	Orges Velia
4	US_S3_03	As a developer, I want the pipeline to be modular and independently testable so that changes can be made without affecting the entire system.	* Pipeline components are modular * Changes do not break other modules * Code is reusable and clean * New features can be added easily * Documentation is updated	High	Security & Resource Optimization	8	Orges Velia
5	US_S3_04	As a data steward, I want every agent action to be logged with execution details so that pipeline runs can be audited.	* All actions are logged with timestamps * Input/output summaries are stored * Logs are retrievable * Audit trail is complete * Logs are structured and readable	High	Traceability & Audit Logging	8	Kush

Sprint 3 Stories Completed

	No.	Description	Acceptance Criteria	Priority	Feature	Story point	Assignee
7	US_S3_06	As a user, I want to validate dataset quality so that I can detect issues before processing.	* Missing values are detected * Duplicate records are identified * Data types are validated * Validation summary is generated * Errors are clearly reported	Medium	Data Quality & Health Insights	8	Priya Prajapati
8	US_S3_07	As a user, I want to rate pipeline runs and provide feedback so that the system can improve over time.	* Users can submit ratings * Feedback is stored successfully * Comments can be added * Feedback is linked to runs * UI is easy to use	Medium	User Feedback & Learning Loop	5	Shreya Kuvalekar
9	US_S3_08	As a user, I want to connect a database to the system so that I can ingest structured data.	* Database connection is successful * Data can be fetched correctly * Connection errors are handled * Data is validated * Logs capture interaction	High	Data Source Integration	8	Orges Velia
10	US_S3_09	As a user, I want to connect API-based datasets so that I can ingest external data into the system.	* API connections work correctly * Data is parsed successfully * Errors are handled properly * Data validation is applied * API interactions are logged	Medium	Data Source Integration	8	Tenzing Salaka

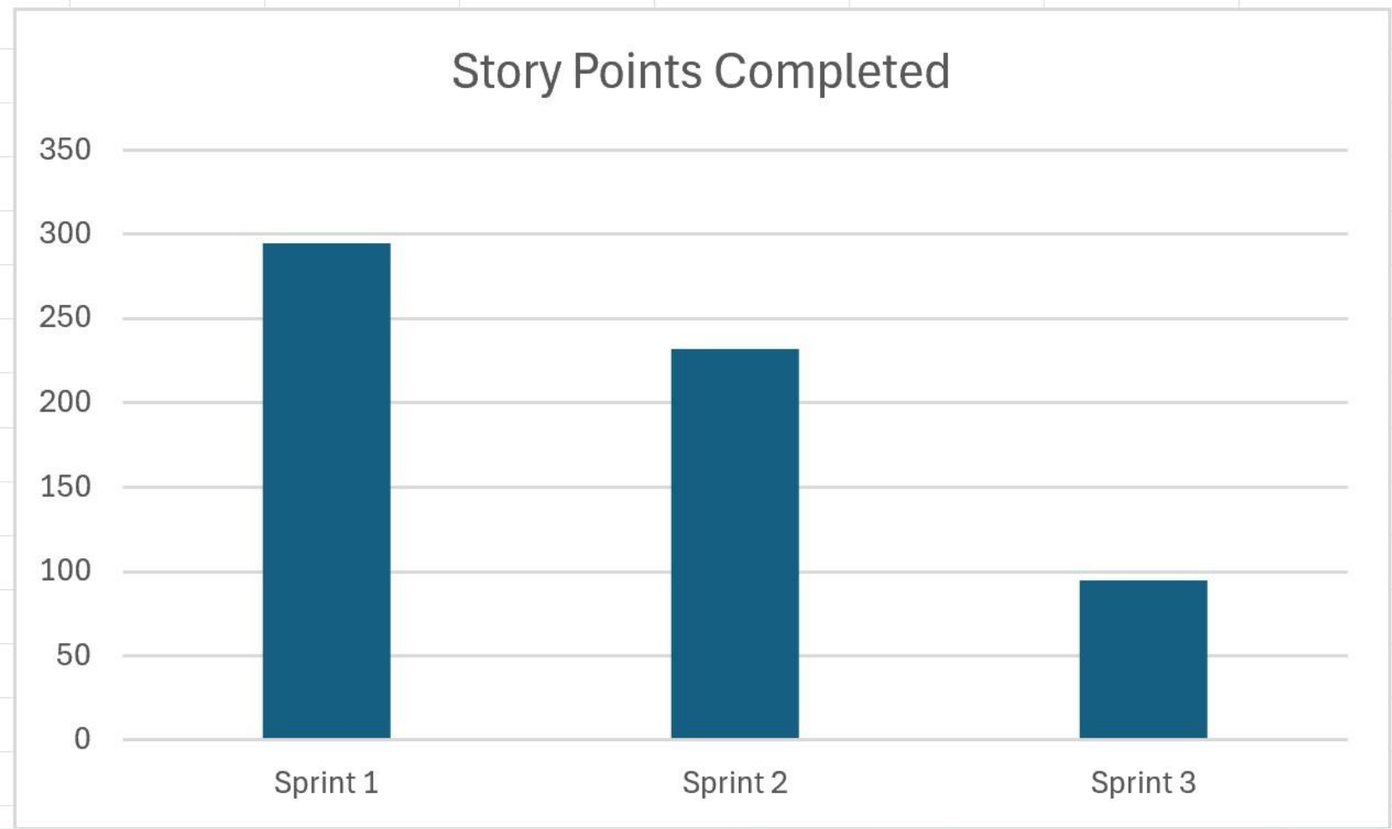
Sprint 3 Test cases

TC ID	Module	Scenario	Priority	Status
TC_S3_01	Pipeline Reliability & Performance	ETL pipeline execution within threshold	High	Pass
TC_S3_02	Pipeline Reliability & Performance	Execution metrics logging	High	Pass
TC_S3_03	Pipeline Reliability & Performance	Bottleneck detection	High	Pass
TC_S3_04	Pipeline Reliability & Performance	Data accuracy after optimization	High	Pass
TC_S3_05	Security & Resource Optimization	No hardcoded credentials	High	Pass
TC_S3_06	Security & Resource Optimization	Sensitive info not logged	High	Pass
TC_S3_07	Security & Resource Optimization	Secure credential storage	High	Pass
TC_S3_08	Security & Resource Optimization	Restricted credential access	High	Pass
TC_S3_09	Security & Resource Optimization	Modular pipeline components	High	Pass
TC_S3_10	Security & Resource Optimization	No cross-module impact	High	Pass
TC_S3_13	Traceability & Audit Logging	Agent actions logging	High	Pass

Sprint 3 Test cases

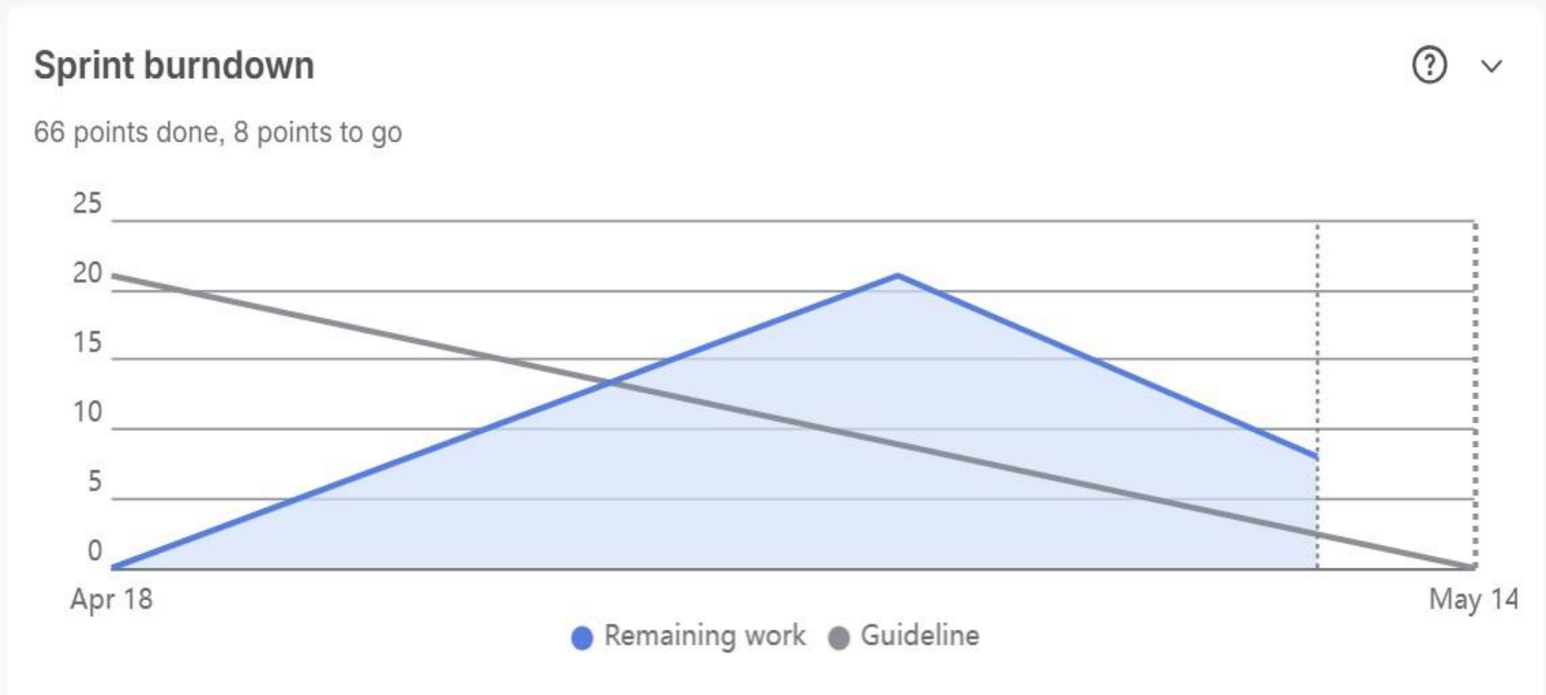
	TC ID	Module	Scenario	Priority	Status
13	TC_S3_14	Traceability & Audit Logging	Input/output summaries stored	High	Pass
14	TC_S3_15	Traceability & Audit Logging	Log retrieval	High	Pass
15	TC_S3_16	Traceability & Audit Logging	Audit trail completeness	High	Pass
16	TC_S3_18	Pipeline Reliability & Performance	Large dataset handling	High	Pass
17	TC_S3_19	Pipeline Reliability & Performance	Performance under load	High	Pass
18	TC_S3_20	Pipeline Reliability & Performance	No crashes under heavy processing	High	Pass
19	TC_S3_31	Data Source Integration	Database connection success	High	Pass
20	TC_S3_32	Data Source Integration	Structured data fetching	High	Pass
21	TC_S3_33	Data Source Integration	DB error handling	High	Pass

Team Velocity



Sprint 3

Burndown Chart



Sprint 3

Completed/Committed Ratio

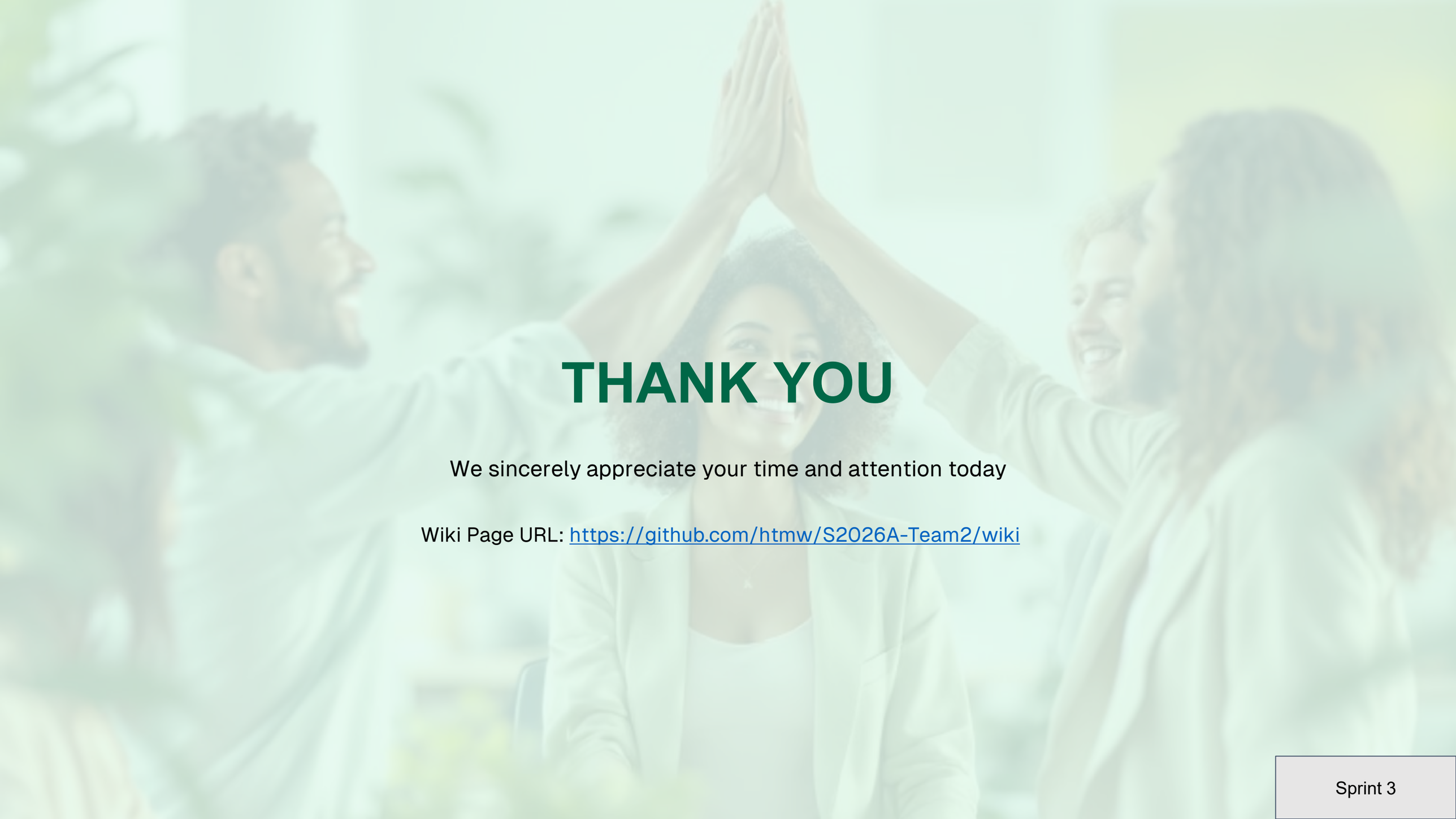
Sprint 3

Sprint 3 Retrospective

- **What went well**
  - All committed work was completed (**100% completion**)
  - Strong delivery in **pipeline performance, security, and audit logging**
  - System stability improved with better handling of workloads and integrations
- **What to improve**
  - **Velocity dropped** compared to earlier sprints
  - Less focus on **medium-priority features** like data quality and user feedback
  - Need better **workload distribution and planning accuracy**
- **Action items**
  - Improve **estimation and sprint planning**
  - Balance focus between **core and enhancement features**
  - Increase **testing, especially for scalability**
  - Enhance **documentation and user-facing features**

Sprint 3 Demo

Demo Video



THANK YOU

We sincerely appreciate your time and attention today

Wiki Page URL: https://github.com/htmw/S2026A-Team2/wiki